

# Demostración de ECDH en Python

## 1. Introducción

El protocolo ECDH (Elliptic Curve Diffie-Hellman) permite que dos partes acuerden un secreto compartido utilizando criptografía de curvas elípticas.

En aplicaciones modernas se usa **Curve25519**, implementada como X25519 en la librería `cryptography` de Python.

## 2. Instalación

```
pip install cryptography
```

## 3. Ejemplo completo de ECDH con X25519

**Curve25519** es una curva elíptica diseñada por Daniel J. Bernstein en 2005 con el objetivo explícito de ofrecer alta seguridad, eficiencia y resistencia a errores de implementación.

Está definida sobre el cuerpo primo

$$\mathbb{F}_p \quad \text{con} \quad p = 2^{255} - 19,$$

lo que permite implementaciones muy eficientes en software debido a la forma especial del primo.

En su forma original (Montgomery) la curva está dada por:

$$y^2 = x^3 + 486662x^2 + x$$

sobre  $\mathbb{F}_p$ . Las claves tienen 256 bits de longitud (el número de puntos racionales es aprox.  $2^{256}$ ).

En aplicaciones prácticas de intercambio de claves se utiliza la variante **X25519**, que trabaja únicamente con coordenadas  $x$ . Esto reduce la complejidad, mejora el rendimiento y disminuye la superficie de ataque frente a implementaciones defectuosas.

## Nivel de seguridad

- Proporciona aproximadamente **128 bits de seguridad** (ver el punto siguiente). Es equivalente a claves RSA de 3072 bits, pero con un rendimiento mucho mayor.
- El mejor ataque conocido es el método de Pollard–rho, con complejidad aproximada  $O(2^{128})$  (cantidad de operaciones).  
En general, su complejidad es aproximadamente  $O(\sqrt{n})$ , donde  $n$  es el orden del grupo (en esta curva el orden del grupo es aprox  $2^{252}$ ).  
En términos prácticos:

$$2^{128} \approx 3,4 \times 10^{38}$$

Este número es astronómicamente grande. Incluso si un atacante pudiera probar  $10^{12}$  claves por segundo (mucho más rápido que cualquier sistema realista), necesitaría:

$$\frac{2^{128}}{10^{12}} \approx 10^{26} \text{ segundos}$$

lo cual supera enormemente la edad del universo.

- No se conocen ataques prácticos contra el logaritmo discreto en esta curva.
- Fue diseñada para evitar vulnerabilidades presentes en curvas NIST.
- Tiene resistencia incorporada frente a ciertos ataques de canal lateral.

## Uso en la práctica

Curve25519 (X25519) es utilizada en:

- TLS 1.3

- Signal
- OpenSSH
- WireGuard

Actualmente es una de las curvas recomendadas para nuevos desarrollos criptográficos debido a su combinación de seguridad comprobada, simplicidad estructural y alto rendimiento.

```
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

# =====
# 1. Generación de claves
# =====

# Alice genera su clave privada
alice_private_key = x25519.X25519PrivateKey.generate()
alice_public_key = alice_private_key.public_key()

# Bob genera su clave privada
bob_private_key = x25519.X25519PrivateKey.generate()
bob_public_key = bob_private_key.public_key()

# =====
# 2. Cálculo del secreto compartido
# =====

alice_shared_secret = alice_private_key.exchange(bob_public_key)
bob_shared_secret = bob_private_key.exchange(alice_public_key)

print("¿Coinciden los secretos?",
      alice_shared_secret == bob_shared_secret)

# =====
# 3. Derivación de clave simétrica (recomendado)
# =====

derived_key = HKDF(
```

```

        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake data',
    ).derive(alice_shared_secret)

print("Clave derivada (hex):", derived_key.hex())

```

Salida:

¿Coinciden los secretos?: True

Clave derivada (hex): f919170d829d7007d7bc1deea3ffa061f35a1e264baf1541cf1cd5efe76aa85a

## 4. Otro completo de ECDH con X25519

```

from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives import serialization
import base64
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes
import os

# Generate a private key for Alice
alice_private_key = X25519PrivateKey.generate()
alice_public_key = alice_private_key.public_key()
print("\nSe imprime la clave publica en formato RAW (crudo)\n")
raw_pub_alice = alice_public_key.public_bytes(
    encoding=serialization.Encoding.Raw,
    format=serialization.PublicFormat.Raw
)
print(raw_pub_alice.hex())

# Generate a private key for Bob
bob_private_key = X25519PrivateKey.generate()
bob_public_key = bob_private_key.public_key()

print("\nSe imprime la clave publica en formato PEM (mas amigable)")
pem_pub_bob = bob_public_key.public_bytes(
    encoding=serialization.Encoding.PEM,

```

```

        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    print(pem_pub_bob.decode())

    # Alice calculates the shared secret using her private key and Bob's public key
    shared_secret_alice = alice_private_key.exchange(bob_public_key)

    # Bob calculates the shared secret using his private key and Alice's public key
    shared_secret_bob = bob_private_key.exchange(alice_public_key)

    # The shared secrets should be the same
    assert shared_secret_alice == shared_secret_bob

    # It is best practice to use a Key Derivation Function (KDF) on the shared secret
    # before using it as an encryption key.
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake data',
    ).derive(shared_secret_alice)

    # Use 'derived_key' for symmetric encryption (e.g., AES-GCM or XChaCha20-Poly1305)

    print(type(shared_secret_alice))
    print(len(shared_secret_alice))
    print(type(bob_public_key))
    print(derived_key)

```

Salida:

Se imprime la clave publica en formato RAW (crudo)

```
a84e406baec3ae8b7f45994fc0740adef6f84e85286f2f6832f351dae6dde007
```

Se imprime la clave publica en formato PEM (mas amigable)

```
-----BEGIN PUBLIC KEY-----
```

```
MCowBQYDK2VuAyEAKPj4YbfQV9Eftk3bC//JDnR8+ONZjrP2wE7s5ZZ4+yY=
```

```
-----END PUBLIC KEY-----
```

```
<class 'bytes'>
```

32

```
<class 'cryptography.hazmat.bindings._rust.openssl.x25519.X25519PublicKey'>
b'z\xed\xbd\xde\x0fuF\xd6i\x8c\xb9x\x01u\xaa\xf0\x96\xb1\xfe\xef\x19\x1fI\xde
```

## 5. Cifrado autenticado con AES-GCM

Una vez derivada la clave simétrica con HKDF, podemos usarla para cifrar mensajes de manera autenticada mediante AES-GCM.

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# =====
# 4. Cifrado con AES-GCM
# =====

# Usamos la clave derivada (32 bytes -> AES-256)
aesgcm = AESGCM(derived_key)

# AES-GCM requiere un nonce único de 12 bytes
nonce = os.urandom(12)

mensaje = b"Mensaje secreto usando ECDH + AES-GCM"

# Cifrado
ciphertext = aesgcm.encrypt(nonce, mensaje, associated_data=None)

print("Ciphertext (hex):", ciphertext.hex())

# =====
# 5. Descifrado
# =====

mensaje_descifrado = aesgcm.decrypt(
    nonce,
    ciphertext,
    associated_data=None
)
```

```
print("Mensaje descifrado:", mensaje_descifrado)
```

## Notas de seguridad

- El nonce debe ser único para cada cifrado con la misma clave.
- AES-GCM proporciona confidencialidad e integridad.
- Si el ciphertext es modificado, el método `decrypt()` lanza una excepción.
- En protocolos reales, el nonce se transmite junto con el ciphertext.

## 6. Observación de seguridad

- Nunca se debe usar directamente el secreto crudo.
- Siempre se debe aplicar un KDF (por ejemplo HKDF).
- X25519 está diseñada para resistir ataques de implementación.
- Es el estándar usado en TLS moderno y protocolos como Signal.

## 7. ECDH con secp256r1 (NIST P-256)

Además de Curve25519, otra curva ampliamente utilizada es **secp256r1**, también conocida como **NIST P-256**.

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

# =====
# 1. Generación de claves
# =====

alice_private_key = ec.generate_private_key(ec.SECP256R1())
alice_public_key = alice_private_key.public_key()
```

```

bob_private_key = ec.generate_private_key(ec.SECP256R1())
bob_public_key = bob_private_key.public_key()

# =====
# 2. Cálculo del secreto compartido
# =====

alice_shared_secret = alice_private_key.exchange(
    ec.ECDH(),
    bob_public_key
)

bob_shared_secret = bob_private_key.exchange(
    ec.ECDH(),
    alice_public_key
)

print("¿Coinciden los secretos?:",
      alice_shared_secret == bob_shared_secret)

# =====
# 3. Derivación de clave
# =====

derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=None,
    info=b'handshake data',
).derive(alice_shared_secret)

print("Clave derivada (hex):", derived_key.hex())

```

## 8. La curva secp256r1 (NIST P-256)

La curva **secp256r1**, también conocida como **NIST P-256**, es una curva elíptica estandarizada por el NIST en 1999.

Está definida sobre el cuerpo primo

$$\mathbb{F}_p \quad \text{con} \quad p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1.$$

Es una curva en forma de Weierstrass corta:

$$y^2 = x^3 + ax + b$$

con parámetros específicos publicados por el NIST.

## Nivel de seguridad

- Proporciona aproximadamente **128 bits de seguridad**.
- El mejor ataque genérico conocido es Pollard–rho.
- Complejidad aproximada:  $O(2^{128})$ .
- Equivalente en seguridad simétrica a AES-128.

## Uso en la práctica

- TLS 1.2 y anteriores
- Certificados digitales (ECDSA)
- Infraestructura PKI
- Hardware criptográfico

## Comparación con Curve25519

- Ambas ofrecen 128 bits de seguridad.
- Curve25519 fue diseñada priorizando seguridad de implementación.
- P-256 fue estandarizada por el NIST.
- Curve25519 suele ser más simple de implementar correctamente.
- Ambas se consideran seguras frente a ataques clásicos.